Experiment - 04

Develope dynamic Webpage using javascript

Code:

```
<html>
 <head>
 <title> Random Colour Generator </title>
 <link  ref = "stylesheet" href = "style.css">
 </head>
 <body>
    <h3 id = "colorText"> Generate a Random Color</h3
   <button id = "genBtn"> Generate Color </button>
   <div id = "colorBox"> This is your new color </div>
   <script>
   let btn = document.getElementById("genBtn");
   let h3 = document.getElementById("colorText");
   let box = document.getElementById("color Box");

   btn.addEventListener ("click", function() {
        let ran = genRandColor();
        h3.innerText = ran;
        box.style.backgroundColor = ran;
        console.log ("color updated");
   });
  function genRandColor(){
      let red = Math.floor (Math.random() *255);
      let green = Math.floor (Math.random()*255);
      let blue = Math.floor (Math.random()*255);
  return 'rgb ( ${red}, ${green}, ${blue})';
}
```

```
</script>
</body>
</html>
```

Experiment - 05

05. Implement Modules in Node.js

In Node.js, you can implement modules using two distinct systems: common Js (Js), which is traditional default, and ES modules (ESM), the modern javascript standard

1. The CommonJs Method (default):

CommonJs uses module.exports to share code and require() to import it. This system loads modules synchronously

i) create the Module (math.js):

```
const add = (a,b) => a+b;
const substract = (a,b) => a-b;
module.exports = {
    add,
    subtract
};
```

ii) use the module (app.js):

```
Const math = require ('./math');
console.log (math.add (10,5));
console.log (calculatorName);
console.log (multiply (4,2));
console.log (divide (4,2));
```

o/p: superCalc 15

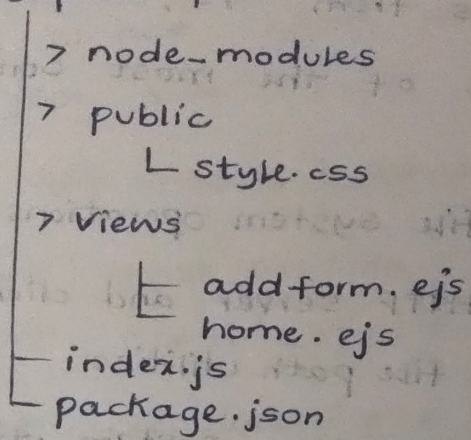
8 5

2

Experiment-06

06. Develope a Mini application using Nodejs and Express js

Folder structure



Codes:

→ index.js

```
Const express = require ('express');
const app = express ();
const bodyParser = require ('body-parser');
app.set ('view engine', 'ejs');
app.use (express.static ('public'));
app.use (bodyParser.urlencoded ({extend:
                    true}));
let recipes = [ ];
app.get ('/', (req, res) => {
        res.render ("home", {recipes});
3)
app.get ('/add', (req, res) => {
        res.render ('addform');
3)
```

```
app.post ('/add', (req, res) => {
    const {title, ingredients, steps} =
        req.body;
    recipes.push ( {title, ingredients, steps});
    res.redirect ('/');
});
app.listen (3000, () => {
    console.log ("server started");
});
```

→ home.ejs

```
<html>
 <head>
    <title> Recipe Book </title>
    <link rel= "stylesheet" href = "style.css">
 </head>
    <h1> Welcome to recipe-book </h1>
    <a href = "/add"> Add recipe </a>
    <ul>
      <% for recipe in recipes {%>
        <li>
          <h2> <%= recipe.title %> </h2>
          <p> <strong> Ingredients : </strong>
              <%= recipe.ingredients %> </p>
          <p> <strong> Steps: </strong>
              <%= recipe.steps %> </p>
        </li>
    <% }%>
      </ul>
</html>
```

→ addform.ejs

```
<html>
<head>
    <title> Add recipe from </title>
    <link rel = "stylesheet" href = "style.css">
</head>
Enter new recipe
<form action = "/add" method = "POST">
    <label> Title = </label> <br>
    <input type = "text" name = "title required
                                    <br>
    <label > Ingredients : </label > <br>
    <textarea name = "ingredients " required
                                    </textarea>
    <br> <label> Steps: </label > <br>
    <textarea name = "steps" required >
                                    </textarea> <br>
    <button type = "submit" > Add Recipe </butt
</form>
</html>
```

→ style.css

```
body {
    font-family: Arial, sens-serif;
    margin: 20px;
}
h1{
    color: #333;
}
a {
    text-decoration : none;
    color: blue;
}
```

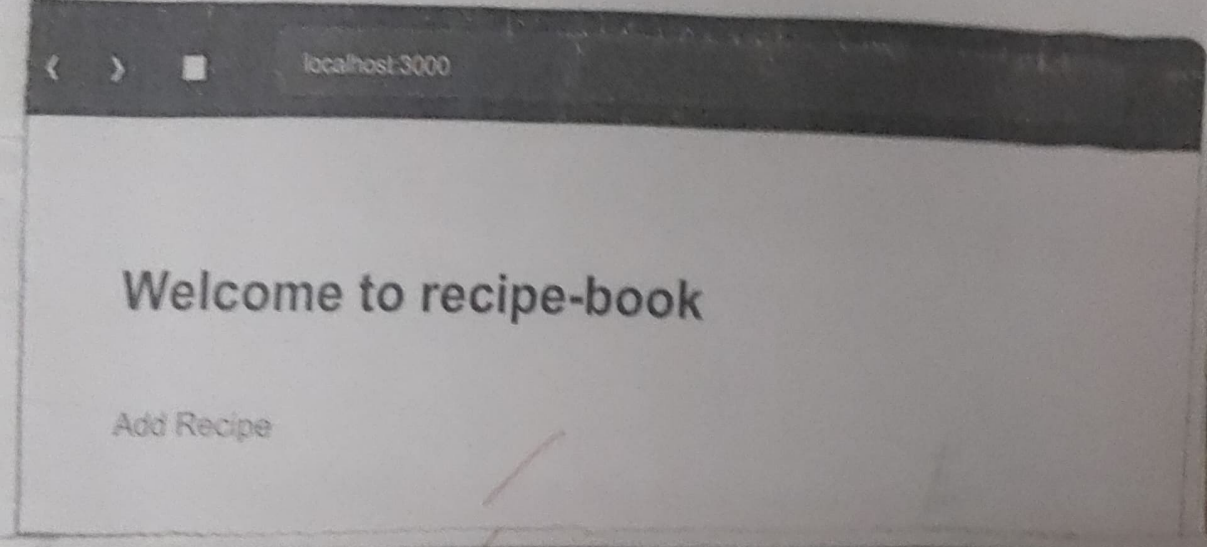
form input, from textarea {

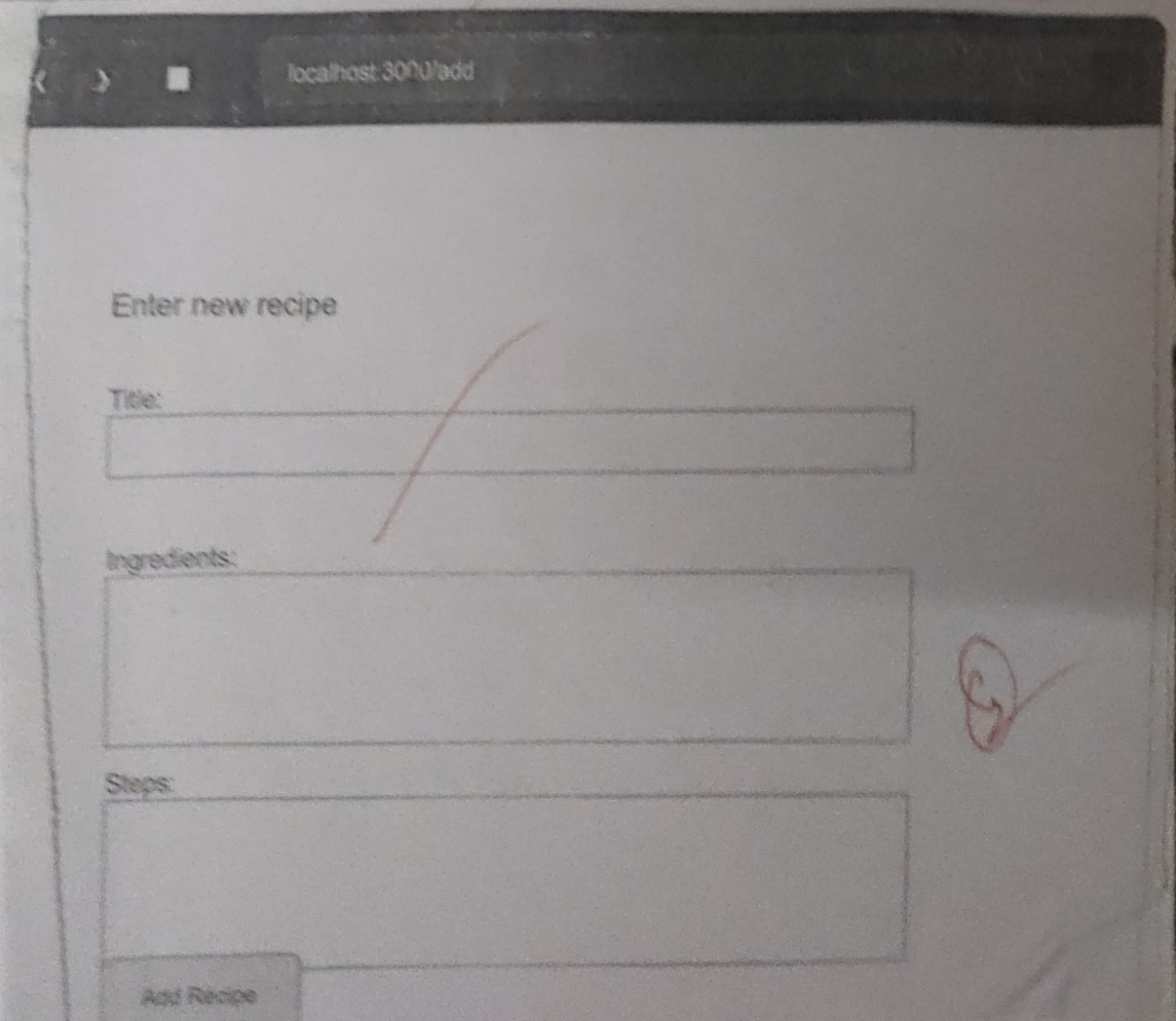
width: 300px;

margin-bottom: 10px;

}

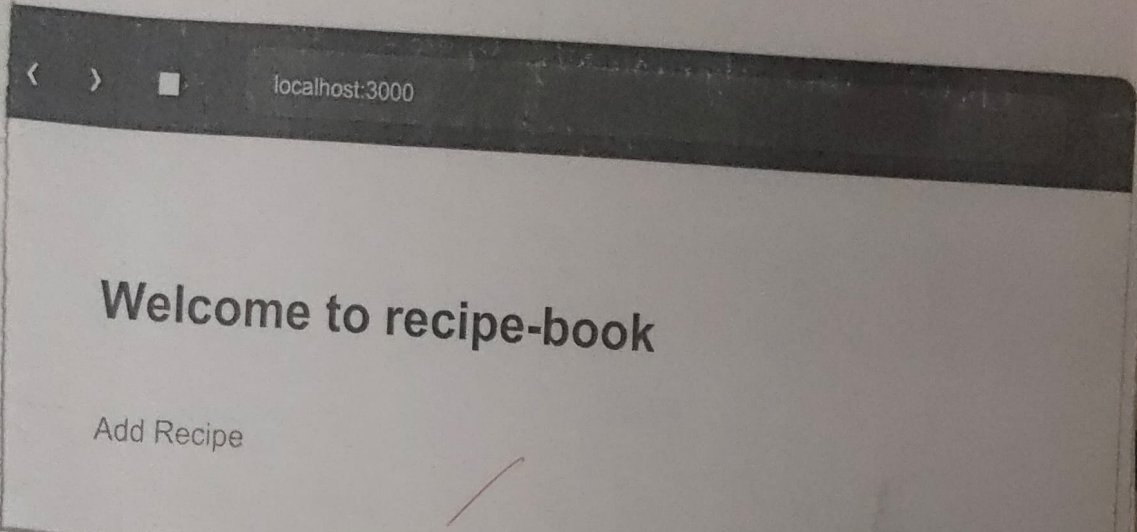
cipe Book — Home UI

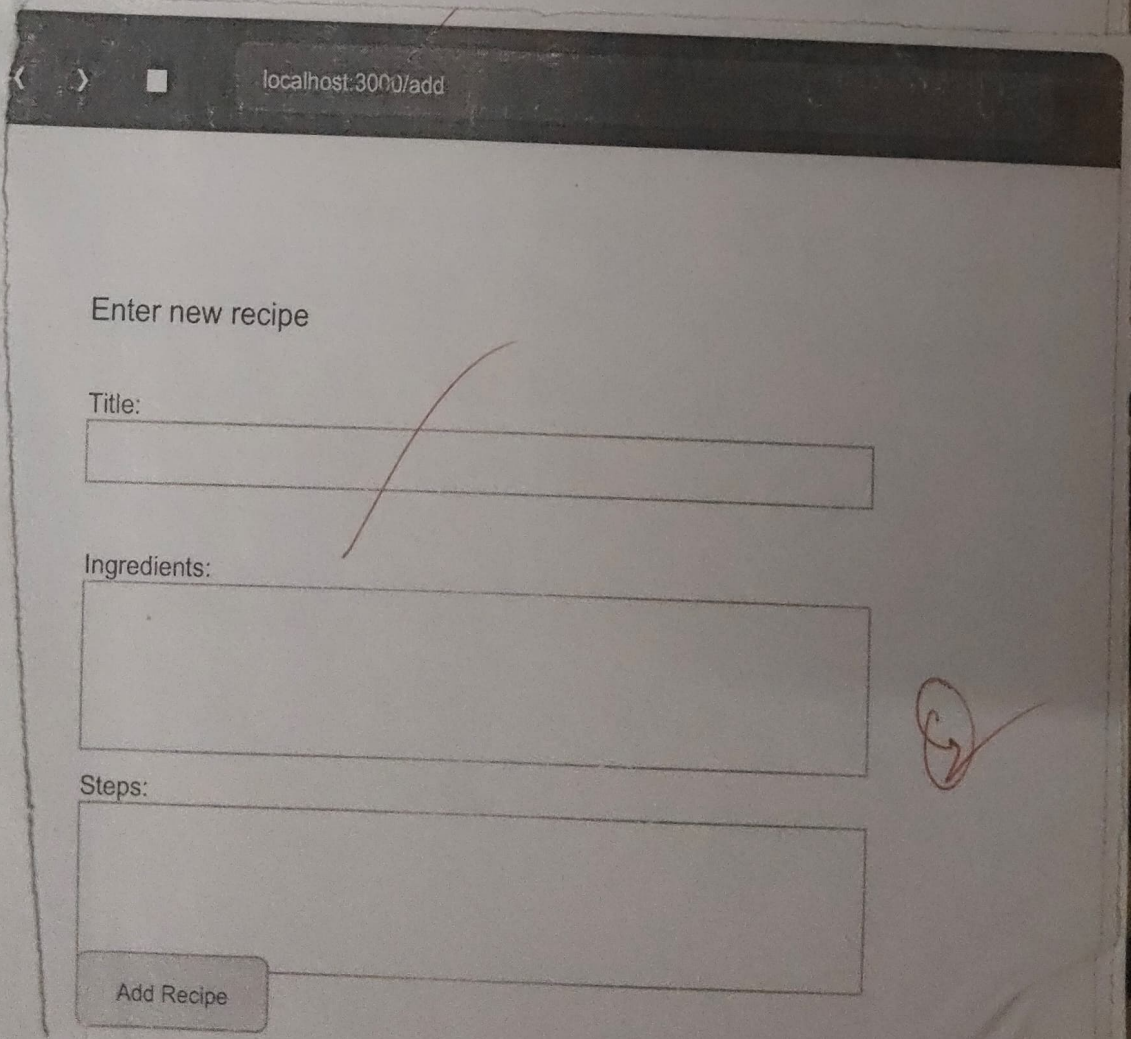




```
form input, from textarea {
    width: 300px;
    margin-bottom: 10px;
}
```

cipe Book — Home UI





7. Create a Functional component and class Compo using react.

Aim: To develope a react application using functio components and class components to display Student details, manage state, handle button clicks. and dynamically update the webpage

Functional Component:

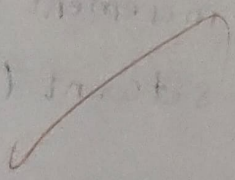
A Functional component is a js function that returns JS r to display the user interface. It uses react hooks such as usestate to manage state and update the webpage dynamically.

Program:

```
import React, {useState} from
'react';
function studentFunctional() {
    const [name, setName] = useState("Harshini");
    const [course, setCourse] = useState("CSM");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Harshini");
    };
```

```
const changeCourse = () => {
    setCourse("CSD");
};
const incrementCount = () => {
    setcount(count+1);
};
return (
  <div style={{
      border = "2px solid blue";
      padding: "20px"
      margin: "20px"
      borderRatio: "10px"
      textAlign : "center"
    }}>
    <h1> Student Details</h1>
    <h2> Functional Component </h2>
    <p> <b> Name : </b> {name} </p>
    <p> <b> Course: </b> {course} </p>
    <p> <b> Button clicks: </b> {count} </p>
    <button onclick = {changeName}>
        changeName </button>
    <button onclick = {changeCourse}change
        course</button>
    <button onclick = {incrementCount}>
        Increment
    </button>
  </div>
);
```

Output: when the user clicks the buttons, the name, Course and Count change dynamically without reloading the webpage



class Component:

A class Component is a ES6 JS class that extends React.Component. It uses the render() method to return JSX and manager state using this.state and this.setState()

Program:-

```
import React, {Component} from
'react';
class studentClass extends Component {
    Constructor(props) {
     super(props);
     this.state = {
          name = "Ch.Harshini",
          course :" Computer Science",
          count: 0
     };
    }
    changeName = () => {
          this.setstate {{
          name: "Harshini"
     });
    };
    changeCourse = () => {
     this.setstate ({
     course: "CSM" });
    };
```

```
incrementCount = () => {
    this.setState((prevState) => ({
        count: prevState.count + 1
    }));
};
render() {
    return (
        <div style={{
            border: "2px solid green";
            padding: "20px";
            margin: "20px";
            borderRadius: "10px";
            textAlign: "center"
        }}>
        <h1>Student Details</h1>
        <h2>class component</h2>
        <p><b>Name:</b> {this.state.name}</p>
        <p><b>Course:</b> {this.state.course}</p>
        <p><b>Button clicks:</b> {this.state.count}</p>
        <button onClick={this.changeName}>
            Change Name</button>
        <button onClick={this.changeCourse}>
            change course</button>
        <button onClick={this.incrementCount}>
            Increment</button></div>
```

Q(b):-

Experiment-05

3. Implement http methods using JSON server

Aim:- To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, update, Delete) operations on student records.

a) Create a JSON server:

step1: Initialize the project

Open the vscode terminal and execute the following commands:

```
mkdir JSON Server
cd JSON server
npm init -y
npm install json server.
```

step2:- Create db.json

Create a file named db.json and add following student data.

```
{ "Students": [
    {"id" : "1",
     "name" : "Harshini",
     "course": "CSM",
     "marks": '90' },
  { "id" : "2"
     "name": "Thanmayee",
```

"course" : "csm",

"marks" : 90

},

{ "id" : "3"

"name" : "Rebekah",

"course" : "IT",

"marks" : 88

}

Step 3 : Start the JSON server

for the current. JSON server version.

execute:

hpx json. server db.json

The server start at

http: // local host : 3000

The student records can be accessed with;

http: // localhost: 3000 / students.

2. Implementation of HTTP methods

a. GET Method:- The GET method is used to retrieve student records from JSON server

code:-

```
fetch ('http://localhost:3000/students')
  .then (response => response.json())
  .then (data) => {
      console.log(data);
  })
  .catch (error => {
      console.log(error);
  });
```

o/p:-

B. Post Method: The Post Method is used to insert a new student record into server

code:-

```
fetch ("http://localhost:3000/students", {
    method = 'post',
    header = {
    'content-Type': 'application/json'
    },
    body: JSON.stringify ({
        name: 'Rahul',
        course: 'CSE',
        marks: 92
    })
})
.then (respons => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {
    console.log (error);
});
```

c. PUT METHOD: The put Method is used to replace on existing student record with updated data

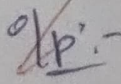
Code:

```
fetch ('http://localhost:3000/student
    method: 'PUT',
    header: {
        'content-Type': 'application/j
    },
    body: JSON.stringify({
        name: 'Harshini',
        Course: 'AIML',
        Marks: 95
    })
})
.then (response => response.json())
.then (data => {
    console.log(data); })
.catch (error => {
console.log (error)
});
```

E. DELETE Method:- The delete Method is used to remove an existing student record from server

code: fetch ('http://localhost:3000/students/3', {
method: 'DELETE' })
.then (response => {
if (response => o{
consofe.log ("student deleted succesfully");

3
3)
.catch (error => {
console.log(error);
3);



After executing the POST, PUT, PATCH, and DELETE operations in sequence, the remaining student will be:

O/P:-

Experiment-09

9. Develop CRUD operations using MongoDB

Aim:- To develop a web Application, using Node.js, Express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student record

1. Installation and project setup:

```
mkdir mongoDB.CRUD
cd MongoDB.CRUD
npm init.y
npm install express Mongoose
```

Program: server.js

```
const express. require ('express');
const mongoose = require ('mongoose');
Const app = express ();
const PORT = 3000;
app. use (express.json());
mongoose. connect ('mongobd://127.0.0.1:
                    27017/college")
.then ((1 ⇒ console.log ("mongo DB connected")
.catch (err ⇒ console.log(err));
```

```
// create Student
const StudentSchema = new
mongoose.Schema({
    name: {
    type: String,
    required: true
    },
    marks: {
    type: Number
    required: true
    }
});
// create student model
const student =
    mongoose.model("Student", studentSchema)

// create Add
app.post('/students', async((req, res)
                              =>{
    try{
const student = new Student(req.body);
const saved student = await student.save()
res.status(201).json(saveStudent);
} catch (error){
```

```
// create Student
Const Student Schema = new
mongoose.schema ({
    name: {
    type: string,
    required: true
    },
    marks: {
    type: Number
    required: true
    }
});
// create student model
Const student =
    mongoose.model ("Student", studentSchema);

// create Add
app.post ("/students", asign ((req, res)
                              => {
    try{
const student = new_student (req.body);
Const saved student = await student.save();
res.status (201).json (.serveStudent);
} catch (error) {
```

```
res. status(400). json ({message:
                error.message});
    }
});

res. status(200). json(student);
} catch (error){
  res.status (400). json { message:
  error. message};
}
});
// update
app. put ('/students/:id', async (req, res)
                                  => {
  try{
  const student = await
  Student. findByIdAndUpdate(
  req. params. id,
  req. body,
  {new: true, runValidators: true}

  );
  if (!student) {
    return res. status (404). json({
      message: "student not found".
  });
```

//DELETE

```
app.delete('/students/:id', async (req, res) => {

  try {
    const student = await student
        findByIdAndDelete(
        req.params.id);

    if (!student) {
      return res.status(404).json({
      message: "student not found"

    });

  }
  res.status(200).json({
    message: 'student deleted successfully'
  });
  } catch (error) {
    res.status(400).json({message:
                error.message});

  }
});
```